



Université de
Sherbrooke

Université de Sherbrooke

SQL

Aperçu

SQL_00

Christina KHNAISSER (christina.khnaisser@usherbrooke.ca)

Luc LAVOIE (luc.lavoie@usherbrooke.ca)

(les auteurs sont cités en ordre alphabétique nominal)

CoFELI/Scriptorum/SQL_00-Apercu, version 1.0.0.a, en date du 2026-01-11

— document de travail, ne pas citer —

Plan

Introduction	3
1. Présentation	4
2. Concepts par l'exemple.	9
Conclusion	33
Références.	34
Définitions	35

Introduction

Le présent document a pour but de présenter une vue générale du langage de programmation SQL.

1. Présentation

Structured Query Language (SQL) est un langage de programmation standardisé servant à définir et exploiter des bases de données relationnelles.

SQL a été créé au début des années 1970 par IBM (ALPHA).

Dans l'année 1980, *l'American National Standards Institute* (ANSI) a commencé à travailler pour standardiser le langage SQL pour permettre la portabilité des applications.

La norme ISO/IEC 9075^[1] est l'une des normes les plus importantes des bases de données relationnelles (4544 pages).

- Première version : 1987
- Dernière version : 2023
- Versions majeures : 1992, 1999, 2003, 2008, 2011, 2016, 2019

SQL est un langage déclaratif qui consiste à décrire le problème à résoudre et non comment le résoudre (le comportement).

1.1. Modèle vs. SQL

Tableau 1. Éléments de base

Modèle	SQL	Définition
Relation	TABLE	Un prédicat modélisé par un ensemble de tuples représenté par une table.
Tuple	ROW	Une proposition modélisée par un ensemble d'attributs représentée par une ligne.
Attribut	COLUMN	Une caractéristique typée et valuée désignée par un identifiant.
Type de base	TYPE	Un ensemble des valeurs propres.
Sous-type	DOMAIN	Un sous-ensemble d'un type.
Base de données	DATABASE	Un ensemble de variables correspondant à un modèle logique.

Nous ajouterons le concept de **Schéma** qu'on retrouve dans plusieurs SGBD. Le schéma est un conteneur pour les éléments relationnels, ce qui permet de faciliter l'organisation des relations et la gestion des accès.

1.2. Organisation du langage

- Langage de définition de données (LDD)
 - Schéma (SCHEMA)
 - Tables (TABLE)
 - Vues (VIEW)
 - Types (TYPE, DOMAIN)
 - Opérateurs (OPERATOR)
 - Fonctions (FUNCTION)
 - Procédures (PROCEDURE)
 - Automatismes (TRIGGER)
 - ...

- Langage de manipulation de données (LMD)
 - insertion des données (INSERT)
 - mise à jour des données (UPDATE)
 - retrait des données (DELETE)
 - évaluation des données (SELECT)
- Langage de contrôle de données (LCD)
- Langage de transaction de données (LTD)

2. Concepts par l'exemple

2.1. Les prédicats

Activité

L'activité identifiée par le sigle «sigle», décrite par le titre «titre», est offerte par l'UdeS.

Étudiant

L'étudiant identifié par le matricule «matricule», décrit par le nom «nom» habite à l'adresse «adresse», est inscrit à l'UdeS.

TypeEvaluation

Le type d'évaluation de identifié par le code «code», décrit par la description «description», est autorisé à l'UdeS.

Résultat

- Le résultat «note» a été obtenu par l'étudiant dont identifié par le matricule est «matricule» lors de l'évaluation «TE» dans le cadre de l'activité «activite» au trimestre «trimestre».

2.2. Les données

UNIVERSITÉ

Étudiant

<u>matricule</u>	nom	adresse
15113150	Paul	>Δ ^ς σ▷ ^ς b
15112354	Éliane	Blanc-Sablon
15113870	Mohamed	Tadoussac
15110132	Sergeï	Chandler

Activité

<u>sigle</u>	titre
IFT159	Analyse et programmation
IFT187	Éléments de bases de données
IMN117	Acquisition des médias numériques
IGE401	Gestion de projets
GMQ103	Géopositionnement

TypeÉvaluation

<u>code</u>	description
IN	Examen intra
FI	Examen final
TP	Travail pratique
PR	Projet

Résultat

<u>matricule</u>	<u>TE</u>	<u>activité</u>	<u>trimestre</u>	note
15113150	TP	IFT187	20133	80
15112354	FI	IFT187	20123	78
15113150	TP	IFT159	20133	75
15112354	FI	GMQ103	20123	85
15110132	IN	IMN117	20123	90
15110132	IN	IFT187	20133	45
15112354	FI	IFT159	20123	52

2.3. Les types

Tableau 2. Exemples de types prédéfinis (PostgreSQL)

Rel	SQL	Définition
Texte	Varchar	Chaîne de caractères de longueur variable.
Entier	Integer	Entier entre de -2147483648 à +2147483647.
Booléen	Boolean	État « true » (vrai), « false » (faux).

$\text{SigleCours} \{ x \in \text{Texte} \mid x \sim [\text{A-Z}]\{3\}[\text{0-9}]\{3\} \}$

Chaine de caractères dont les trois premiers caractères sont des lettres latines majuscules et les trois derniers des chiffres.

```
create domain SigleCours  
  varchar  
  check(value similar to '[A-Z]{3}[0-9]{3}');
```

Matricule $\{ x \in \text{Texte} \mid x \sim [0-9]\{8\} \}$

Chaine de caractères dont les caractères sont des chiffres.

```
create domain Matricule  
  Varchar  
  check(value smiliar to '[0-9]{8}');
```

$\text{TypeEval} \{ x \in \text{Texte} \mid x \in \{\text{TP}, \text{PR}, \text{IN}, \text{FI}\} \}$

Un code de type d'évaluation composé de deux lettre en majuscule.

```
create domain TypeEval  
  Varchar  
  check(value in ('TP', 'PR', 'IN', 'FI'));
```


$\text{Note} \{ x \in \text{Entier} \mid 0 \leq x \leq 100 \}$

Entier positif entre 0 et 100.

```
create domain Note  
  SmallInt  
  check (value between 0 and 100);
```

Trimestre { $t \in \text{Texte} \mid t \sim [0-9]\{4\}[1-3]$ }

Chaine de caractères encodée en suffixant le chiffre du trimestre à une année postérieure à 1927 (année de fondation de l'UdeS). Chiffre associé au trimestre : hiver $\rightarrow$ 1, été $\rightarrow$ 2, automne $\rightarrow$ 3.

```
create domain Trimestre  
  Varchar  
  check(value similar to '[0-9]{4}[1-3]');
```

2.4. Les tables

Activité {sigle: SigleCours; titre: Texte}

```
create table Activite
(
    sigle SigleCours not null,
    titre varchar      not null
);
```

Étudiant {matricule: Matricule; nom: Texte; adresse: Texte}

```
create table Etudiant
(
  matricule Matricule not null,
  nom       varchar   not null,
  adresse   varchar   not null
);
```

TypeÉvaluation {code: TypeEval; description: Texte}

```
create table TypeEvaluation  
(  
  code          TypeEval not null,  
  description varchar  not null  
);
```

Résultat {matricule: Matricule; TE: TypeEval; activité: SigleCours; trimestre: Trimestre; note: Note}

```
create table Resultat
(
  matricule Matricule not null,
  TE        TypeEval  not null,
  activite  SigleCours not null,
  trimestre Trimestre not null,
  note      Note       not null
);
```

2.5. Les contraintes

2.5.1. Les clés primaires

Activité {sigle: SigleCours; titre: Texte}
clé { sigle }

```
create table Activite
(
  sigle SigleCours not null,
  titre varchar      not null,
  constraint Activite_cc0 primary key (sigle)
);
```

Étudiant {matricule: Matricule; nom: Texte; adresse: Texte}
clé {matricule}

```
create table Etudiant
(
  matricule Matricule not null,
  nom       varchar    not null,
  adresse   varchar    not null,
  constraint Etudiant_cc0 primary key (matricule)
);
```


TypeÉvaluation {code: TypeEval; description: Texte}
clé {code}

```
create table TypeEvaluation  
(  
    code          TypeEval not null,  
    description varchar not null,  
    constraint TypeEvaluation_cc0 primary key (code)  
);
```

Résultat {matricule: Matricule; TE: TypeEval; activité: SigleCours; trimestre: Trimestre; note: Note}
clé {matricule, activite, TE, trimestre}

```
create table Resultat
(
  matricule Matricule not null,
  TE        TypeEval  not null,
  activite  SigleCours not null,
  trimestre Trimestre not null,
  note      Note       not null,
  constraint Resultat_cc0 primary key (matricule, activite,
TE, trimestre)
```

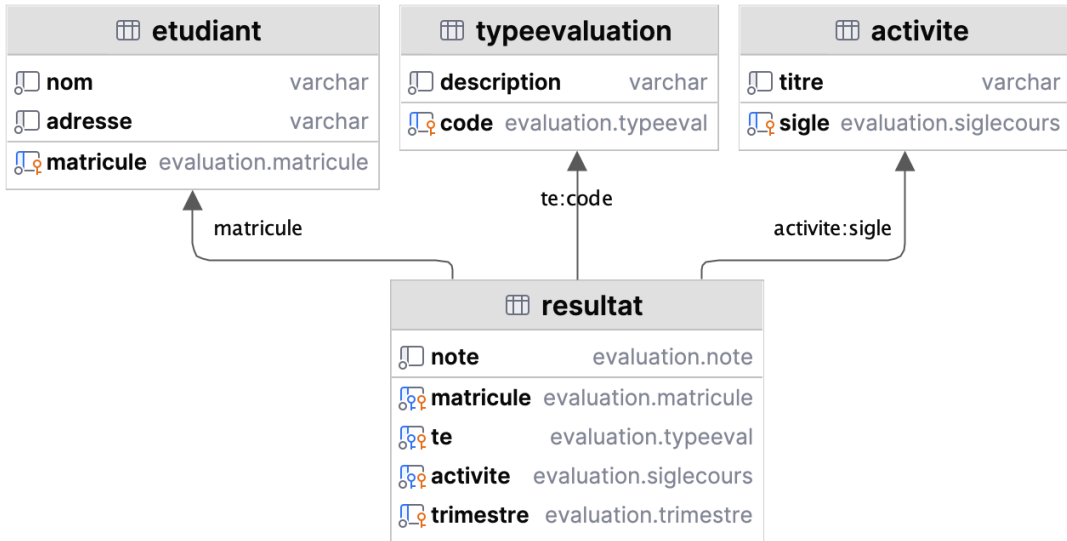
);

2.5.2. Les clés référentielles

Résultat {matricule: Matricule; TE: TypeEval; activité: SigleCours; trimestre:
Trimestre; note: Note}
cle(matricule, activite, TE, trimestre)
matricule → Étudiant.matricule
TE → TypeÉvaluation.code
activité → Activité.sigle

```
create table Resultat
(
    matricule Matricule not null,
    TE         TypeEval  not null,
    activite   SigleCours not null,
    trimestre Trimestre not null,
    note       Note       not null,
    constraint Resultat_cc0 primary key (matricule, activite,
TE, trimestre),
    constraint Resultat_cr0 foreign key (matricule) references
Etudiant (matricule),
    constraint Resultat_cr1 foreign key (activite) references
Activite (sigle),
    constraint Resultat_cr2 foreign key (TE) references
TypeEvaluation (code)
);
```

2.6. Le diagramme relationnel



Conclusion

Références

PG

[fr] <https://docs.postgresql.fr/18/index.html> (2026-01-05)

[en] <https://www.postgresql.org/docs/18/index.html> (2026-01-05)

PG - Correspondance de motif

[fr] <https://docs.postgresql.fr/18/functions-matching.html#FUNCTIONS-SIMILARTO-REGEXP> (2026-01-11)

PG - Conformité SQL

[fr] <https://docs.postgresql.fr/18/features.html> (2026-01-11)

Définitions

SGBDR (Système de gestion de bases de données relationnelles)

SGBD utilisant un modèle d'abstraction fondée sur la théorie relationnelle de Codd et intégrant comme critère de qualité le maintien des propriétés ACID.

SQL (*Structure Query Language*)

Langage de programmation axiomatique fondé sur un modèle inspiré de la théorie relationnelle proposée par E. F. Codd.

[Normes applicables : ISO 9075:2016, ISO 9075:2023]

Produit le 2026-01-13 16:52:54 UTC



Université de Sherbrooke

[1] « L'Organisation internationale de normalisation (en anglais : International Organization for Standardization - ISO) est un organisme de normalisation international composé de représentants d'organisations nationales de normalisation de 171 pays, selon le principe d'un membre par pays. » (https://fr.wikipedia.org/wiki/Organisation_internationale_de_normalisation)